Python Calculator (calculator.py) Walkthrough

A beginner-friendly architectural guide to how your code functions step-by-step.

Line Number / Code Snippet	What It Does Under the Hood

`Lines 1-2:`
`def add(a, b):`
`return a + b`
	Defining an Addition Function: This sets up a specific math engine named 'add'. The slots (a, b) take the two values you provide later. The return driver delivers the arithmetic sum back to the screen.

`Lines 4-5:`
`def sub(a, b):`
`return a - b`
	Defining a Subtraction Function: Operates exactly like addition, but performs subtraction (a minus b) and hands the difference back to the script execution block.

`Lines 7-8:`
`def mul(a, b):`
`return a * b`
	Defining a Multiplication Function: Built to process products. It multiplies the contents of the parameter values and outputs the final calculation result.

`Lines 10-14:`
`def div(a, b):`
`if b == 0:`
`return "Error: ..."`
`else:`
`return a / b`
	The Division Safety Shield: Standard computers will instantly throw a crash error if forced to divide by zero. This conditional check (if b == 0) intercepts a 0 input and outputs a message string safely instead of crashing your workflow.

`Lines 16-19:`
`def show_menu():`
`print("\nSimple ...")`
	Menu Renderer: Prints out your operational instructions in the console window so you know what actions to choose next during runtime.

`Lines 22-30:`
`def get_number(...):`
`while True:`
`try:`
`...`
`except ValueError:`
	Safe Value Validator: Uses a try/except guard layer. If you accidentally type standard text words instead of floating point numbers, it flags a ValueError and prompts you again instead of breaking out.

`Lines 32-38:`
`def main():`
`while True:`
`...`
`if op == "exit":`
`break`
	The Infinite Runtime Application Loop: The while True command constructs a continuous runtime path so you can process multiple calculations in succession. Typing 'exit' triggers the break statement, dropping out of the calculation loops cleanly.

Essential Development Core Loop Routine

Whenever you make modifications to your logic, remember to run these three terminal lines sequentially:

1. `git add calculator.py (Staging changes into the directory buffer)`
2. `git commit -m "Your summary note" (Locking in a safe history timestamp snapshot)`
3. `git push origin main (Blasting your update directly into your GitHub cloud repository)`

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.